



School of Engineering and Science

General code Overview

First Online Activity

Project

Blazar Variability

Resumed by

Ezequiel Gómez Valencia A01667568

August 12, 2026

1 Introduction and Initial Considerations

This document provides a general overview of the data treatment performed by doctoral student *Miguel Ángel Chávez Robles* to do a time series analysis with PROPHEET. An appendix has been included at the very end of this work to explain some relatively important functions used in the code shared via *Notion*.

This document isn't focused on explaining each code line of September's efforts. However, it is a great tool that the *Next-Gen Scientists 2026 Summer Team* has created to help future young researchers catch up quickly and target specific questions.

To understand specific concepts about the PROPHEET documentation, please, before reading this, go check the documents we have uploaded at **Blazar Research > Prophet > General Prophet Overview**. Once you've completed this, you may start reading the following descriptions and clarifications.

NOTE: *To visualize the corresponding code, please contact the group leader to have your access granted.*

2 Before Using Prophet

There are some things we have to understand before trying to make predictions with *PROPHET*. The data usually undergoes a specific cleaning process in Python (often using the pandas library). Some specific points of information that usually have to be changed are listed below:

1. **Column Renaming:** *Prophet* is strictly hardcoded to look for two specific columns.
2. **Dataframe Formatting:** Date-time objects must be standardized in specific formats like YYYY-MM-DD or YYYY-MM-DD HH:MM:SS.
3. **Outlier and NaN Management:** Even though Prophet is robust against missing data and outliers, good data science practice involves cleaning these up. Many notebooks contain steps capping extreme values or explicitly dropping empty rows so the trend line isn't artificially skewed.

3 Archives Used for Prophet Predictions

3.1 Manual Table Event Filtering

This *.xlsx* document type contains the information that serves as a manual event tracker. Before trying to analyse the information contained inside the time series, human domain experts recorded everything that physically happened to the telescope and its environment.

The column description is the following:

- **Index:** A numerical identifier used to identify the corresponding event number.
- **Date:** The corresponding day, month, and year in which the phenomenon occurred. You may notice that some entries correspond to a single day, while others represent a range of days. For this reason, the Python script described in the following sections requires a custom date-parsing function.
- **Type:** The broad category of the event. They have abbreviations that stand for the following.
 - **HW:** Hardware (camera repairs, mirror alignments, dehumidifier)
 - **SW:** Software (for example: mirror alignment)
 - **AT:** Atmospheric or environmental issues.
- **Change:** A small description of the changes that were made or stuff that happened during a specific period of time.
- **Comments:** Additional context.
- **Influence on rate:** An assessment of how the event impacted the system's performance. Things like 'positive step'

3.2 SQL Type Data Base

3.2.1 Time related variables

The columns contained in this archive track when and for how long the telescope or system was observing.

- ***x*:** The primary date of observation. It acts as the main time index.

- ***start* & *stop***: The exact date and time when the system began and finished recording for the night.
- ***ontime***: This variable indicates the total active duration of the observation.

3.2.2 Measurements and Statistic-related Variables

The following variables contain the actual numerical values captured by the system. They are typically used in forecasting models like those in PROPHET.

- ***y***: The value calculated for that date.
- ***err_y***: The uncertainty associated with the y measurement.

3.2.3 Positional & System Variables

- ***admin* & *zdmx***: These variables represent the minimum and maximum Zenith Distance, respectively.
- ***azmin* & *azmx***: These variables represent the minimum and maximum Azimuth angles, respectively.

4 General Process Followed for the Time Series Prediction

After importing the required libraries, the general process can be divided into four main stages: data cleaning and event mapping, quality control and filtering, time-series analysis and seasonality, and finally forecasting and model evaluation.

4.1 Data Cleaning and Event Mapping

The first stage consists of preparing the information obtained from both the manual event log and the observational database. Since the manual table contains information about physical events that may have influenced the telescope's performance, it is necessary to identify which events should be incorporated into the forecasting model.

4.1.1 Filtering the Manual Log

The author loads *manual_table_filtering.xlsx* Excel file and removes the *Index* column, since this one is only used as a numerical identifier and does not provide relevant

information for the prediction.

The, the table is filtered using the *keep* column. Only events explicitly marked as '*Yes (mirror)*' or '*Yes (dust)*' are retained. This filtering process allows the model to focus only on events that were considered relevant to the telescope's performance.

4.1.2 Custom Regular Expression Date Parsing

One of the main difficulties during the data-cleaning process is the interpretation of the dates contained in the manual event table. Some events correspond to a single day, while others represent a range of days.

For this reason, the custom *parse_date_range* function was modified to correctly interpret specific German-style date formats. In particular, a new regular expression is introduced:

```
r'(\d{1,2})\.-(\d{1,2})\.(\\d{1,2})\.(\\d{4})$'
```

Which was designed to identify date ranges occurring within the same month. Correctly interpreting these ranges is important because the duration of an event is later used to define its influence on the prediction.

4.1.3 Structuring Prophet Features

Once the relevant events have been identified and their dates have been parsed, they are transformed into specific features that can be incorporated into *Prophet*.

- **Holidays (Dust):** Dust events are incorporated into the model as holidays. The duration of each event is calculated by subtracting the *start_date* from the *end_date*. This duration is then assigned to the *upper_window* parameter. A dataframe is created in which the event is identified as '*TNGDust*'.
- **Changepoints (Mirrors):** Mirror alignment events are treated differently. Instead of representing them as holidays, the author extracts only their corresponding *start_date* and stores these dates in a list called *changepoints*. These dates are later incorporated into the forecasting model as potential points where the underlying trend of the data may change.

4.2 Quality Control and Filtering

After the relevant events have been mapped, a second filtering stage is performed. The purpose of this stage is to remove observations that could introduce biases or inconsistencies into the forecasting process.

4.2.1 Dropping Old Data

Following a note referring to “Daniela’s suggestion,” the author filters both the main dataframe and the event dataframes to retain only observations occurring after September 1, 2013.

This can be understood as a temporal restriction that establishes the beginning of the period considered reliable for the analysis. By applying the same temporal criterion to the observational and event datasets, the different sources remain consistent with one another.

4.2.2 Masking Problematic Periods

Another important step consists of identifying periods in which the observed values may not represent the normal behavior of the system.

Based on the plotted data and notes contained in the manual log, particularly references to “*too high rates*” and changes in container temperature, the author identifies a problematic period between May 16, 2017 and July 4, 2017.

The corresponding dates are defined as:

```
start_date = 2017-05-16
end_date   = 2017-07-04
```

A pandas mask is then used to retain only the observations outside this interval:

```
(df['ds'] < start_date) | (df['ds'] > end_date)
```

Consequently, all observations falling within the problematic period are removed before the model is fitted.

4.2.3 Final Pre-processing

Before fitting the forecasting model, the author performs a final cleaning step. The *err_y* column is removed because the forecasting procedure operates directly on the measured values contained in *y*. Additionally, rows containing missing values are removed using the *dropna()* method.

At this point, the dataframe contains the cleaned observations required by *Prophet*, with the time variable stored in the *ds* column and the corresponding measurement stored in the *y* column.

4.3 Advanced Time-Series Mathematics and Seasonality

Once the observational data has been cleaned, the author performs an additional analysis to identify periodic behavior within the time series. This is particularly important because astronomical observations may contain periodic variations associated with physical phenomena.

4.3.1 The Lomb–Scargle Periodogram

To identify possible periodic behavior in a dataset containing missing observations, the author defines a function called *find_dominant_period_robust*.

The first step consists of converting the observation dates into elapsed time measured in days. This is accomplished by converting the difference between dates into seconds and subsequently dividing by the number of seconds in a day:

```
total_seconds() / (24 * 3600)
```

The resulting time array is then used together with the measured signal. Before calculating the periodogram, the mean of the signal is subtracted from the observations in order to center the data around zero.

The author then uses the *scipy.signal.lombscargle* function to calculate the strength of periodic signals across a predefined grid of angular frequencies. This method is particularly useful for astronomical datasets because it can identify periodic behavior even when the observations are not evenly spaced.

The frequency associated with the strongest periodic signal is subsequently used to determine the corresponding *dominant_period*.

4.3.2 Prophet Parameters

After identifying the possible periodic behavior in the data, the *Prophet* model is initialized with a series of specific parameters.

First, the weekly seasonality is explicitly disabled:

```
weekly_seasonality=False
```

This prevents the model from automatically introducing a weekly pattern that may not have physical significance for the observational dataset.

The author also defines the *changepoint_prior_scale* parameter using a predefined constant called *PRIOR*. In this case, its value is:

PRIOR = 0.03

This parameter controls how flexible the model is when fitting changes in the underlying trend.

4.3.3 Custom Lunar Cycle

In addition to the default seasonal components, the author manually introduces a custom seasonality called *moon_period*. This component has a period of approximately 29.5 days and uses a *fourier_order* of 5.

The corresponding configuration is conceptually represented as:

```
name='moon_period'
period=29.5
fourier_order=5
```

The purpose of this custom seasonality is to allow the model to capture variations associated with the physical lunar cycle rather than relying exclusively on Prophet's pre-defined seasonal components.

4.4 Forecasting, Backtesting, and Residual Analysis

After the data has been cleaned and the relevant seasonal components have been identified, the final stage consists of fitting the model, evaluating its predictive performance, and analyzing the resulting residuals.

4.4.1 Combining Changepoints

Before fitting the final model, the author first runs a preliminary Prophet model in order to obtain the automatically detected changepoints, which are stored in *auto_changepoints*.

These automatically detected dates are then combined with the manually identified mirror alignment dates. The two lists are concatenated, duplicate dates are removed, and the resulting values are sorted chronologically.

The resulting unified list is subsequently passed to the final Prophet model through its *changepoints* parameter. This procedure allows the model to incorporate both statistically detected changes and changes identified from the physical history of the telescope.

4.4.2 Train/Test Split

To evaluate the forecasting performance, the author separates the observational data into a training set and a testing set.

The last three months of observations are reserved for testing. The splitting date is calculated using:

```
split_date = df['ds'].max() - pd.DateOffset(months=3)
```

The observations before this date constitute the training dataframe, *train_df*, while the observations after this date are reserved for evaluating the model.

The Prophet model is fitted exclusively using the training data. Afterwards, predictions are generated for the dates contained in the test set.

4.4.3 Error Calculation

Once the model has generated predictions for the test period, its performance is evaluated using two common regression metrics: Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE).

The MAE measures the average absolute difference between the observed and predicted values. Meanwhile, the RMSE gives greater weight to larger errors because the differences are squared before calculating the mean and taking the square root.

These metrics provide two complementary perspectives on the predictive performance of the model.

4.4.4 Residual Analysis

Finally, the author performs a residual analysis to investigate how the model's predictions differ from the actual observations.

A dataframe called *summary_df* is created containing the actual values, the predicted values, and the corresponding residual. The residual is calculated as:

$$\text{residual} = y_{\text{actual}} - y_{\text{predicted}}. \quad (1)$$

The average residual is then calculated to estimate the overall bias of the model, while its standard deviation provides information about the dispersion of the prediction errors.

To visualize this behavior, the residuals are represented using a histogram. Additionally, a shaded region corresponding to the range of ± 1 standard deviation is added to the

histogram. This allows the author to visually identify how concentrated the residuals are around their mean and to assess the general distribution of the model’s errors.

4.5 General Summary of the Process

In general, the procedure followed by the author can be summarized as a sequence of interconnected stages. First, the raw observational and manual event data are cleaned and standardized. Relevant physical events are then mapped into Prophet-compatible features, with dust events represented as holidays and mirror alignments represented as changepoints.

Afterwards, temporal restrictions and quality-control filters are applied to remove unreliable observations. The cleaned dataset is subsequently analyzed for periodic behavior using the Lomb–Scargle periodogram, while a custom lunar seasonality is incorporated into the Prophet model.

Finally, automatically detected and manually identified changepoints are combined before fitting the final model. The last three months of observations are reserved for backtesting, allowing the predictive performance to be quantified using MAE and RMSE. The analysis is completed by studying the residuals, their mean, standard deviation, and overall distribution.

This workflow combines data cleaning, physical knowledge of the telescope, time-series mathematics, and statistical forecasting into a single procedure. Therefore, the resulting Prophet model is not based exclusively on the historical values of the observational dataset, but also incorporates information about known physical events and periodic phenomena that may influence the system’s behavior.